

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – Enkapsulasi



Farrel Labib Alhaidar

103132400009

S1SD-05-A

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Enkapsulasi dalam Java diterapkan melalui prinsip menyembunyian data (*data hiding*), yaitu melindungi atribut internal kelas menggunakan hak akses *private* dan membukanya secara aman lewat antarmuka *method* *public* berupa *getter* dan *setter* sesuai konvensi penamaan baku. Visibilitas serta perilaku komponen kode dikendalikan oleh *access modifier* (*public*, *protected*, *default*, *private*) dan *non-access modifier* seperti *final* untuk mencegah pewarisan maupun pengubahan nilai, *static* untuk anggota bersama milik kelas, hingga *abstract* dan *synchronized* untuk kontrol konkurensi.

Proses penciptaan dan alokasi memori objek dijalankan oleh konstruktor, yaitu *method* khusus bernama sama persis dengan kelasnya tanpa nilai kembalian (*no return value*) maupun kata kunci *void*. Sebuah kelas dapat memiliki beberapa variasi konstruktor dengan parameter berbeda melalui mekanisme *constructor overloading* untuk mempermudah inisialisasi awal atribut. Seluruh struktur kelas tersebut diorganisasi ke dalam sistem direktori *package* guna menjaga modularitas serta mencegah bentrokan nama via klausa *import*, sembari mematuhi aturan sintaksis yang melarang penggunaan kata-kata kunci baku (*reserved words*) Java sebagai nama variabel.

B. Guided

1. Guided 1

a. Manusia.java

```
package guided.guided1;

public class Manusia {

    // definisi atribut

    private String nama;

    private int umur;

    // constructor

    public Manusia(){};

    public Manusia(String nama){

        this.nama = nama;
```

```

    }

    public Manusia(String nama, int umur){

        this.nama = nama;

        this.umur = umur;

    }

    // method setter

    public void setNama(String a) {

        nama = a;

    }

    public void setUmur(int a) {

        umur = a;

    }

    // method getter

    public String getNama() {

        return nama;

    }

    public int getUmur() {

        return umur;

    }

}

```

b. DemoManusia.java

```

package guided.guided1;

public class DemoManusia {

    public static void main(String[] args) {

        Manusia arrMns[] = new Manusia[3];

    }

}

```

```

        // constructor pertama
        Manusia objMns1 = new Manusia();

        // kedua
        Manusia objMns2 = new Manusia("John");

        // ketiga
        Manusia objMns3 = new Manusia("Bajuri", 44);

        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;

        for (int i = 0; i < 3; i++) {
            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}

```

Output :

```

lasi\project\target\classes" guided.guided1.DemoManusia "
Nama : null
Umur : 0

Nama : John
Umur : 0

Nama : Bajuri
Umur : 44

```

2. Guided 2

a. HargaToken.java

```

package guided.guided2;

public class HargaToken {
    public void info() {

```

```

        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}

```

b. HargaPulsa.java

```

package guided.guided2;

public class HargaPulsa {
    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}

```

c. Main.java

```

package guided.guided2;

public class Main {
    public static void main(String[] args) {
        HargaToken objectToken = new HargaToken();
        objectToken.info();

        HargaPulsa objectPulsa = new HargaPulsa();
        objectPulsa.info();
    }
}

```

Output :

```
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
```

C. Unguided

1. Soal Unguided 1

Mengaudit kelayakan dataset berdasarkan persentase sel kosong (missing value), dengan menerapkan konsep enkapsulasi dan pemisahan package.

Menyimpan data (nama, baris, kolom, missing) secara aman (private), menyediakan 3 variasi constructor, dan otomatis menghitung total dataset yang dibuat via variabel static.

Membuat 3 objek dataset (Titanic, Wine Quality, Iris) dengan 3 cara constructor berbeda, menampungkannya ke array, mencetak laporannya lewat perulangan (looping), lalu menampilkan total dataset yang dibuat (hasilnya: 3).

LaporanDataset.java

```
package unguided.laporan;

import unguided.model.Dataset;
import java.util.Locale;

public class LaporanDataset {

    public void cetak(Dataset dataset) {

        String status = dataset.perluDibersihkan() ? "Perlu dibersihkan"
: "Bersih";

        System.out.println("=== Laporan Dataset ===");
    }
}
```

```

        System.out.println("Nama          : " + dataset.getNama());

        System.out.println("Jumlah Baris : " +
dataset.getJumlahBaris());

        System.out.println("Jumlah Kolom : " +
dataset.getJumlahKolom());

        System.out.printf(Locale.US, "Missing      : %d sel (%.2f%%)\n",
dataset.getJumlahMissing(), dataset.getPersentaseMissing());

        System.out.println("Status          : " + status);

        System.out.println();

    }

}

```

Main.java

```

package unguided.main;

import unguided.model.Dataset;
import unguided.laporan.LaporanDataset;

public class Main {
    public static void main(String[] args) {
        // Objek 1: Constructor tanpa parameter, lalu isi memakai
        // setter
        Dataset dataset1 = new Dataset();
        dataset1.setNama("Titanic");
        dataset1.setJumlahBaris(891);
        dataset1.setJumlahKolom(12);
        dataset1.setJumlahMissing(866);

        // Objek 2: Constructor 1 parameter
        Dataset dataset2 = new Dataset("Wine Quality");

        // Objek 3: Constructor lengkap
        Dataset dataset3 = new Dataset("Iris", 150, 5, 0);
    }
}

```

```

        // Simpan ketiganya dalam array
        Dataset[] daftarDataset = { dataset1, dataset2, dataset3 };

        LaporanDataset laporan = new LaporanDataset();
        for (Dataset d : daftarDataset) {
            laporan.cetak(d);
        }

        System.out.println("Total dataset dibuat : " +
Dataset.getTotalDataset());
    }
}

```

Dataset.java

```

package unguided.model;

public class Dataset {
    // Atribut private (enkapsulasi)
    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;

    // Konstanta
    public static final double BATAS_MISSING = 5.0;

    // Variabel milik class (static)
    private static int totalDataset = 0;

    // Constructor 1: Tanpa parameter
    public Dataset() {
        this.nama = "Tanpa Nama";
        this.jumlahBaris = 0;
        this.jumlahKolom = 0;
        this.jumlahMissing = 0;
        totalDataset++;
    }
}

```

```

// Constructor 2: Hanya parameter nama
public Dataset(String nama) {
    this.nama = nama;
    this.jumlahBaris = 0;
    this.jumlahKolom = 0;
    this.jumlahMissing = 0;
    totalDataset++;
}

// Constructor 3: Lengkap
public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
    this.nama = nama;
    setJumlahBaris(jumlahBaris);
    setJumlahKolom(jumlahKolom);
    setJumlahMissing(jumlahMissing);
    totalDataset++;
}

// Method static untuk mengambil total objek Dataset yang dibuat
public static int getTotalDataset() {
    return totalDataset;
}

// Getter dan Setter
public String getNama() {
    return nama;
}

public void setNama(String nama) {
    this.nama = nama;
}

public int getJumlahBaris() {
    return jumlahBaris;
}

// Nilai tidak berubah jika tidak valid / negatif

```

```
public void setJumlahBaris(int jumlahBaris) {
    if (jumlahBaris >= 0) {
        this.jumlahBaris = jumlahBaris;
    }
}

public int getJumlahKolom() {
    return jumlahKolom;
}

// Nilai tidak berubah jika tidak valid / negatif
public void setJumlahKolom(int jumlahKolom) {
    if (jumlahKolom >= 0) {
        this.jumlahKolom = jumlahKolom;
    }
}

public int getJumlahMissing() {
    return jumlahMissing;
}

// Nilai tidak berubah jika tidak valid / negatif
public void setJumlahMissing(int jumlahMissing) {
    if (jumlahMissing >= 0) {
        this.jumlahMissing = jumlahMissing;
    }
}

// Menghitung persentase missing dibanding total sel (baris x
kolom)
public double getPersentaseMissing() {
    int totalSel = this.jumlahBaris * this.jumlahKolom;
    if (totalSel == 0) {
        return 0.0;
    }
    return ((double) this.jumlahMissing / totalSel) * 100.0;
}

// Mengecek apakah dataset perlu dibersihkan
```

```

    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}

```

Output :

```

target\classes" unguided.main.Main "
=== Laporan Dataset ===
Nama      : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing    : 866 sel (8.10%)
Status     : Perlu dibersihkan

=== Laporan Dataset ===
Nama      : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing    : 0 sel (0.00%)
Status     : Bersih

=== Laporan Dataset ===
Nama      : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing    : 0 sel (0.00%)
Status     : Bersih

Total dataset dibuat : 3

```

Penjelasan :

1. Dataset.java (Model & Logika)

Enkapsulasi, semua atribut dibuat private dan hanya bisa diakses via getter/setter. Setter diberi validasi agar nilai tidak berubah jika diinput angka negatif.

Elemen Static, memiliki konstanta `BATAS_MISSING = 5.0` dan variabel class `totalDataset` yang otomatis bertambah setiap ada objek baru dibuat, lengkap dengan method static `getTotalDataset()`.

Overloading Constructor, menyediakan 3 cara pembuatan objek (tanpa parameter, hanya nama, dan parameter lengkap).

Kalkulasi, menghitung persentase sel kosong dari total sel ($\text{baris} \times \text{kolom}$), lalu method `perluDibersihkan()` mengecek apakah persentase tersebut melebihi 5.0%.

2. LaporanDataset.java (Penyaji Data)

Memiliki method `cetak(Dataset dataset)` yang mengambil data melalui getter. Menentukan status secara dinamis: jika `perluDibersihkan()` bernilai true statusnya "Perlu

dibersihkan", jika false statusnya "Bersih". Menampilkan format laporan dengan persentase 2 digit desimal.

3. Main.java

Menguji 3 cara pembuatan objek: Titanic (constructor kosong lalu diisi setter), Wine Quality (constructor 1 parameter nama), dan Iris (constructor lengkap). Menampung ketiganya ke dalam array Dataset[], lalu mencetak laporannya satu per satu menggunakan perulangan (loop). Menampilkan rekapitulasi total dataset yang dibuat di akhir program memanggil Dataset.getTotalDataset().

D. Kesimpulan

1. Perlindungan Data Melalui *Access Modifier (Data Hiding)*: Enkapsulasi diwujudkan secara nyata dengan menyembunyikan status internal objek, yakni mendeklarasikan atribut/variabel dengan hak akses private agar terlindungi dari intervensi luar langsung, kemudian menyediakan antarmuka publik berupa *method public (getter dan setter)* untuk mengontrol pembacaan dan modifikasi data secara aman.
2. Inisialisasi Instans Objek Menggunakan Konstruktor: Penciptaan objek dan alokasi memorinya ditangani secara terstruktur oleh konstruktor metode khusus yang bernama identik dengan kelasnya dan mutlak tidak memiliki nilai kembalian (*no return value*). Penyiapan nilai atribut saat objek diciptakan dapat diatur secara fleksibel melalui *constructor overloading*.
3. Pengendalian Perilaku dan Modularitas Program (*Modifier & Package*): Karakteristik kelas, metode, maupun variabel dikendalikan secara spesifik menggunakan modifier non-akses (seperti final untuk membatasi pengubahan/pewarisan dan static untuk anggota bersama kelas), serta didukung oleh pengelompokan kelas ke dalam direktori package guna menjaga keteraturan struktur proyek dan mencegah konflik penamaan (*naming conflicts*).